## ⇝ Nebenläufigkeit

↳ Ziele: - Vermeidung ungewollter gegenseitiger Beeinflussung
- Unterstützung gewollter Kooperationen zwischen Prozessen
- Mechanismus zur Synchronisation & Kommunikation

- **Konflikt**

Zwei nebenläufige Prozesse stehen im Konflikt zueinander, wenn es ein Betriebsmittel gibt, das sie gemeinsam benutzen (lesend & schreibend), ansonsten heißen sie unabhängig.

- **Zeitkritischer Ablauf (race condition)**

Ein zeitkritischer Ablauf ist eine Folge von Lese/Schreib-Operationen von im Konflikt stehenden Prozessen, bei denen die Konsistenz der Endzustände der Variablen abhängig von der zeitlichen Reihenfolge der Lese/Schreib-Operationen sind.

- **Wechselseitiger Ausschluss (Mutual Exclusion)**

Ein Verfahren zum wechselseitigen Ausschluss verhindert, dass zu einem Zeitpunkt mehr als ein Prozess lesend oder schreibend auf ein gemeinsam benutztes Betriebsmittel zugreift.

- Kritischer Abschnitt (critical section)

Ein Programmteil - Folge von Anweisungen - in dem auf gemeinsam benutzte Datenbereiche zugegriffen wird.

- Atomare Anweisungsfolge (atomic)

Eine Anweisungsfolge heißt atomar oder unteilbar, wenn ihre Abarbeitung nicht unterbrochen werden kann

↳ Ein einzelner Maschinenbefehl wird immer atomar ausgeführt

↳ $x = x + 1$ ↯ nicht atomar

↝ Anforderungen an Mutual Exclusion:

→ Korrektheit: Zu jedem Zeitpunkt darf sich in jedem kritischen Abschnitt nur ein Prozess befinden

→ Fortschrittsbedingung: Prozesse, die sich nicht im kritischen Abschnitt befinden, dürfen andere Prozesse nicht daran hindern in den kritischen Abschnitt einzutreten.

→ Fairness: Alle Prozesse werden gleich behandelt

→ Verhungern (Starvation): Kein Prozess darf unendlich lange auf den Eintritt in kritischen Abschnitt warten müssen

# BS - HVB

~> Lock: public interface Lock {
    void lock();
    void unlock();
}

~> 1. Sperrung von Interrupts
- lock(): Disable Interrupts
- unlock(): Enable Interrupts
-> Lösung zwar korrekt, aber Übertragbarkeit wird unnötig reduziert

~> 2. Sperrvariablen
- Jeder k.A. bekommt Sperrvariable (haben in shared memory)
- Prozess prüft Sperrvariable vor Eintritt in k.A.
- Zugriff auf Sperrvariable muss atomar sein !!!
  -> Falls nicht ist es möglich, dass zwei Prozesse in k.A. sind
- Alternative: Striktes Alternieren
  -> Korrekt aber Fortschrittsbedingung kann verletzt werden

# BS-UVB

## => Peterson Algorithmus

```
volatile int turn;                                  // Verhindert Deadlock
volatile boolean[] taken = {false, false};          // Sichert Mutex
```

**P1:**

```
void lock(){
    taken[0] = true;
    turn = 1;
    while(turn == 1 & taken[1])
        ;
}
void unlock(){
    taken[0] = false;
}
```

**P2:**

```
void lock(){
    taken[1] = true;
    turn = 0;
    while(turn == 0 & taken[0])
        ;
}
void unlock(){
    taken[1] = false;
}
```

=> Gegenseitige Blockierung wird verhindert

↳ Beide werden nie gleichzeitig !

# BS - WS VB

→ Lösung mit Sperrvariablen
↳ erfordert atomaren Zugriff auf Sperrvariable
↳ Unterstützung durch HW

→ Test-and-Set: Instruktion
- Unteilbares Lesen mit anschließendem Schreiben.

```
bool testAndSet(bool * target) {
    bool rv = *target;
    *target = true;
    return rv;
}
```

```
→ void lock() {
    while (testAndSet(&taken));
}

→ void unlock() {
    taken = false;
}
```

- Spin Lock
Ein Spin Lock ist eine auf Test-and-Set basierende Sperrvariable mit aktivem Warten.

→ Nachteil: - Aktives Warten!
- Verlangen Möglichkeit ☹

→ Vorteil: - Sinnvoll, wenn Warten auf Freigabe der Sperre kürzer als Kontextwechsel ist

# BS - KW3

→ Ticket Spin Lock

- garantieren, dass der k. A. in der Reihenfolge zugegriffen wird, in der der Zugang beantragt wird.

→ Bisher Aktives Warten

↳ ↯ belegt CPU ohne einen Fortschritt zu erzielen

↳ Alternative in Prozesszustand Blocked übergehen

(+) Prozessor ist frei für andere Aktionen

(-) Kosten für zwei Prozesswechsel

• Mutex

Ein Mutex ist ein Mechanismus (Synchronisierungsobjekt) zur Gewährleistung von Mutual Exclusion.

↳ implementiert Interface Lock & kann Zustände owned/not owned annehmen.

↳ lock(): → Versucht Besitzer zu werden

→ Falls schon im Besitz eines anderen Threads, dann wird aufrufender Thread blockiert

↳ unlock(): → Gibt Besitz eines Mutex frei, sofern Aufrufer auch Besitz

→ Ein blockierter Thread, welche am diesem Mutex blockieren, dann wird einer dieser Threads neuer Besitzer

```
[...] pthread_mutex_lock(&m)
[...] pthread_mutex_unlock(&m)

pthread_mutex_t m;

main:
    pthread_mutex_init(&m, NULL)
```

# BS - HÜB

- Condition Variable (Bedingungsvariable)

  Eine Condition Variable ist ein Synchronisationselement, welches Threads erlaubt solange zu blockieren, bis eine bestimmte Bedingung erfüllt ist oder ein bestimmtes Ereignis eintritt.

↳ Condition Variablen werden mit Mutexen verknüpft. Dabei wird beim Warten auf eine Bedingung eine Sperre zu einem davon verknüpften Mutex freigegeben.

→ pthread_cond_init()

→ pthread_cond_wait() blockiert aufrufenden Thread bezüglich einer Condition-Variable

→ pthread_cond_signal() weckt einen beliebigen Thread bzgl. einer Condition-Variable auf.

# BS - KVB

- Monitor

Ein Monitor ist ein Konzept zur einfachen Entwicklung konfliktfreier Programme, welches in eine Programmiersprache eingebettet ist.

→ Monitor: Codeabschnitt mit mehreren Prozeduren, Variablen
↳ Idee: In jedem Monitor eines Programms befindet sich gleichzeitig immer nur ein Prozess
↳ Code zur Sicherstellung des Mutual Exclusion wird durch Compiler erzeugt.

- Semaphore als Datenstrukturen

Ein Semaphore ist eine Datenstruktur zur Verwaltung beschränkt verfügbarer Ressourcen, auf die mehrere Prozesse oder Threads unter Vermeidung von aktivem Warten sicher zugreifen sollen.

↳ Idee: Anstatt gesamten kritischen Abschnitt mit Lock/Unlock-Paar zu sperren, wird nur Code zum Ein-/Austritt in k.A. gesperrt.

- Semaphore (Schnittstelle)

Ein Semaphore regelt den Zugang zu einer mehrfach vorhandenen Ressource
→ Für den Zugang ruft Prozess die blockierende Funktion acquire() aus
→ Nach Nutzung ruft Prozess die Funktion release() aus

→ acquire & release sind atomar!
↳ Beide Aufrufe können von unterschiedlichen Prozessen durchgeführt werden.

# BS-HÜB

→ Struct Semaphore {
    int count;  // Zählervariable
    Queue queue;  // Process queue
}

- Binäre Semaphore, Zählsemaphore

Semaphor, deren Zählvariable nur die Werte 0,1 annehmen kann, heißen binäre Semaphore, sonst Zählsemaphore.

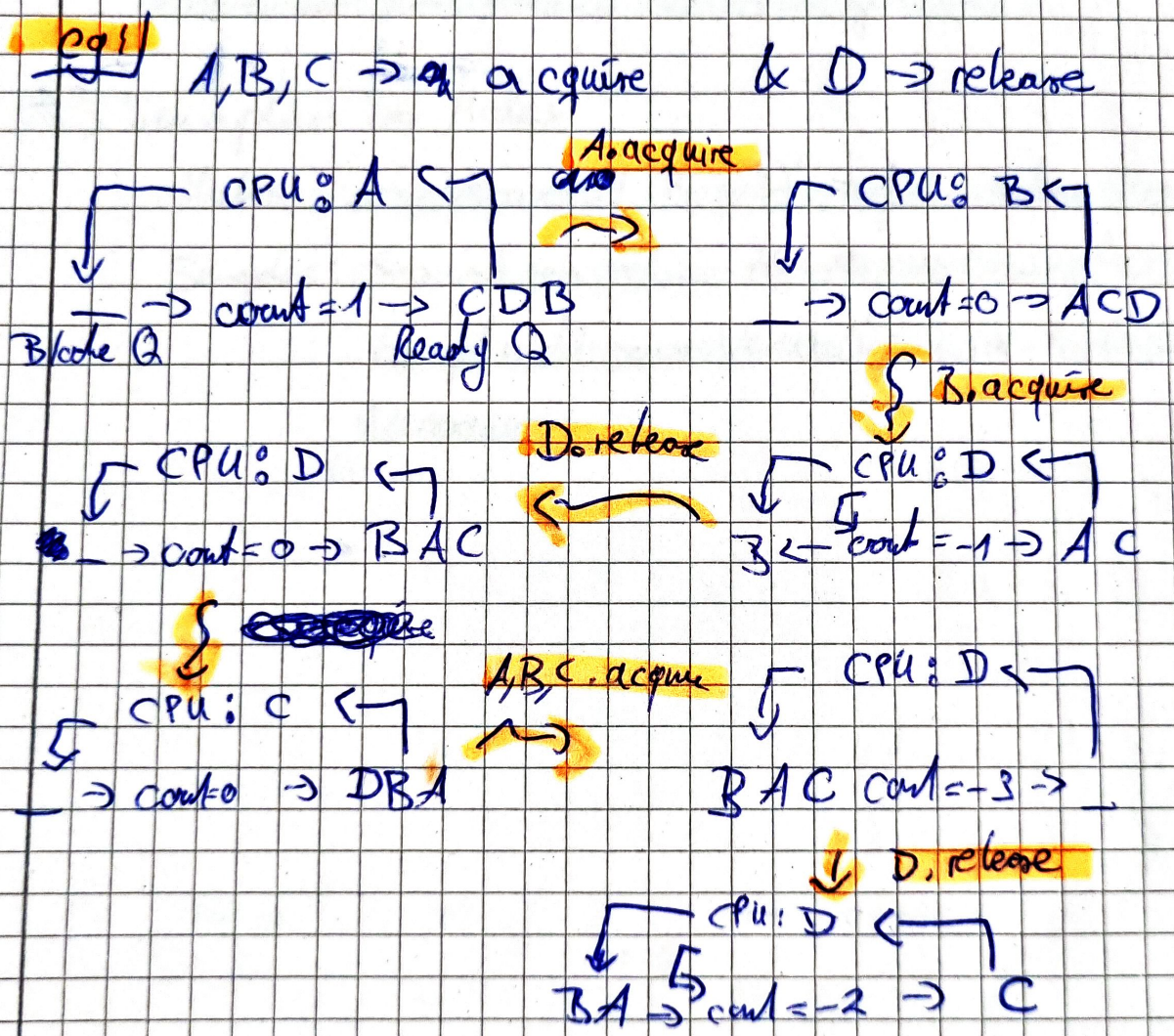
→ Zählsemaphore:
- Zählvariable kann beliebigen Wert annehmen
- Bedeutung von Count:
  - → initial count ≥ 0
  - → count > 0: count Prozesse können acquire() aufrufen ohne zu blockieren
  - → count == 0: der nächste Prozess, der acquire() aufruft, blockiert
  - → count < 0: - count blockierte Prozesse in Queue
- acquire() dekrementiert Wert des Semaphores, bei negativem Wert wird ausführender Prozess blockiert.
- release() inkrementiert Wert des Semaphores, ist Wert nicht positiv wird einer der blockierten Prozesse freigegeben. (in ready Queue einfügen)

→ Kommunikation von Ereignissen zw Prozessen
→ Regelt Zugriff auf eine mehrfach vorhandene Ressource

# BS - VL 03

↝ Starkes Semaphore: Echte Queue nach FIFO-Prinzip

↝ Schwaches Semaphore: Semaphore garantiert nicht die chronologisch richtige Abarbeitung der Queue, sondern ist an anderen Kriterien gebunden




↝ Implementierung von Semaphoren als System calls

# BS KVB

→ Mutex vs Binäres Semaphore

- Mutex gehört einem Thread bzw. Prozess
- Nur der Prozess, der Mutex besitzt, kann Mutex wieder frei geben
- Mutex nur für Mutual Exclusion gedacht!

→ Semaphor vs Mutex

Mutex: → Verwaltet Zugriff auf einzelne Ressource

Semaphore: → ist ein reiner Zähler ⇒ Semaphore allein taugt nicht zur Verwaltung von n-fach vorhandenen Ressourcen.

# BS - K IV/B

→ Erzeuger / Verbraucher:

```
public class Queue {

    private int count = 0;
    private ReentrantLock lock = new ReentrantLock();
    private Condition full = lock.newCondition();
    private Condition empty = lock.newCondition();

    public void add(Item i) {
        lock.lock();
        while (count == N)
            full.await();          // wait till queue not full

        count++;
        empty.signal();            // send non-empty signal
        lock.unlock();
    }

    public void remove() {
        lock.lock();
        while (count == 0)
            empty.await();

        count--;
        full.signal();
        lock.unlock();
    }
}
```

# BS - ÜB

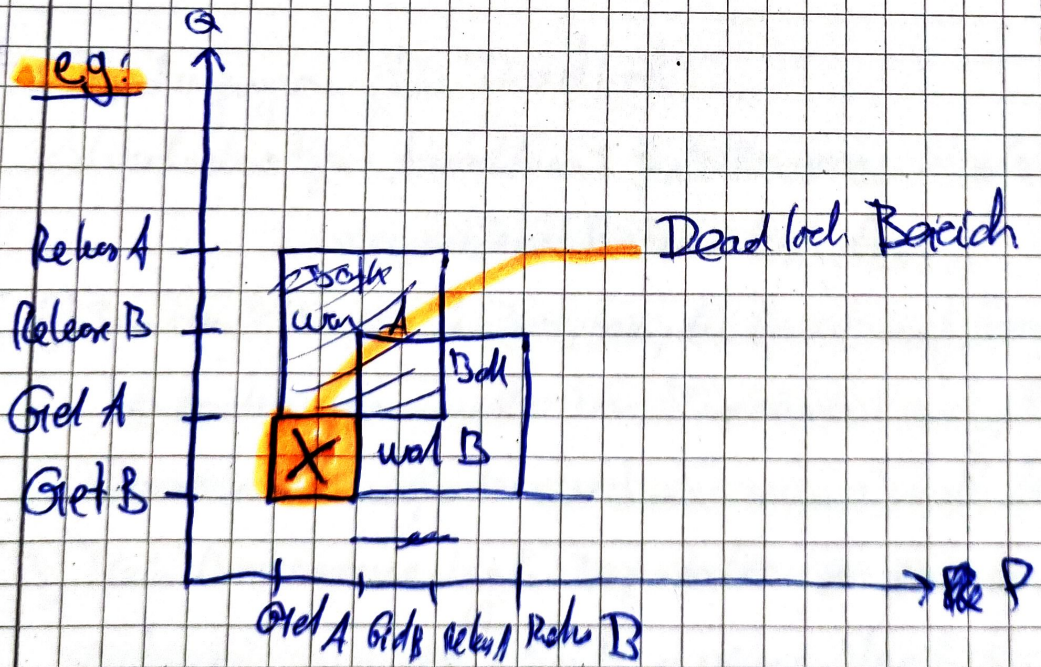
- **Deadlock**

  Eine Menge M von Prozessen sind in einem Deadlock, falls
  → Jeder Prozess aus M auf die Freigabe von Betriebsmitteln wartet, die ein anderer Prozess aus M exklusiv belegt,
  → Jeder Prozess aus M eine Ressource besitzt, auf deren Freigabe ein anderer Prozess aus M wartet

↳ Es gibt verbrauchbare & wiederverwendbare Ressourcen

→ **Joint Progress Diagramm**

- x/y-Achse: Fortschreiten von P/Q
- Ausführung von P & Q wird durch Pfad dargestellt



# BS - K VB

→ Deadlock: Passives Warten & Zustand Blocked

→ Livelock: Aktives Warten & Wechsel Ready & Running

- **Livelock**
  Ein Livelock ist eine Form der Blockierung mehrerer Prozesse, die im Unterschied zum Deadlock nicht in einem Zustand verharren, sondern ständig zwischen mehreren Zuständen wechseln, aus denen sie nicht mehr entkommen können.

- **Verhungern (Starvation)**
  Prozesse die dauerhaft (bzw. sehr lange) warten, ohne dass ein Deadlock vorliegt, nennt man verhungerte Prozesse.

→ Bedingungen für Deadlock:

1) Wechselseitiger Ausschluss: Jede Ressource ist entweder frei oder genau ein Prozess belegt sie
2) Besitzen & Warten: Prozesse, die bereits auf Grund früherer Anforderungen Inhaber von Ressourcen sind, können weitere Ressourcen anfordern und bis zum Erhalt blockieren
3) Kein Ressourcenentzug: Zugeteilte Ressourcen können nicht entzogen werden, sondern müssen durch den Inhaber explizit freigegeben werden.
4) Zyklische Wartebedingung: Es existiert zyklische Kette aus mindestens zwei Prozessen, so dass jeder Prozess eine Ressource anfordert, die vom nächsten Prozess in der Kette belegt ist.

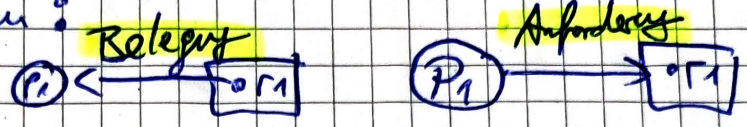
# BS - KUB

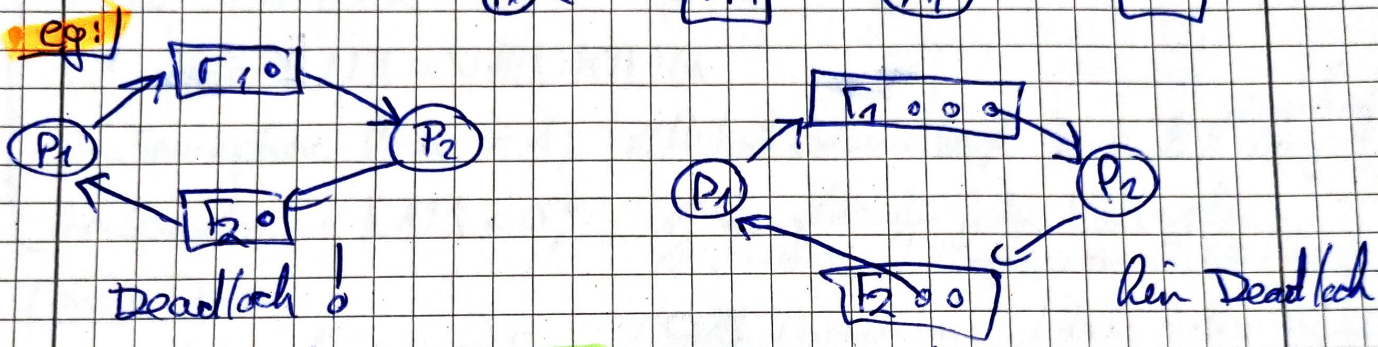
↝ Betriebsmittelbelegungsgraph:

⇒ beschreiben aktuellen Systemzustand

→ Kanten ≙ Prozesse, Betriebsmittel ≙ Rechteck

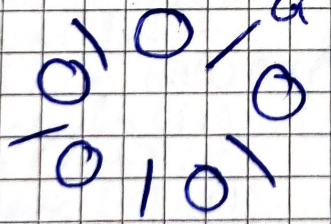
→ Kanten zeigen:




⇒ Bei einfach vorhandenen Ressourcen sind Prozesse genau dann in einem Deadlock verwickelt, wenn sie auf einem Kreis des Graphen liegen.

↝ Dining Philosophers Problem:



↝ P benötigt zwei Gabeln zum Essen.

↝ P Denken ↭ Essen

Deadlock: Alle greifen gleichzeitig die rechte Gabel

Livelock: Wenn gegriffen wird ist linke frei ist.

# BS - KVB

## ↪ Lösung Dining Philosophers mit Semaphoren

Global:

```
#define N 5 // #P
#define RIGHT(i) ((N+i-1)%N)   // Nummer des rechten Nachbarn von i
#define LEFT(i)  ((i+1)%N)     // -"- linken -"-

#define DENKEN 0
#define HUNGRY 1
#define ESSEN 2

int state[N] = DENKEN
Bin Semaphore lock = 1;  // Mutual Exclusion bzgl state & // Zugriff auf Array state
Bin Semaphor s[N] = 0;   // Semaphor für jeden Philosophen
                         // i-tes P ist blockiert, wenn Gabeln nicht verfügbar
```

main()

```
void philosophes(int i){
    while(true){
        think();
        take_forks(i);   // Greife Gabeln or Block
        eat()
        put_forks(i);
    }
}
void take_forks(int i){
    acquire(lock);
    state[i] = HUNGRY;
    check_forks(i);      // Versuche beide Gabeln zu nehmen
    release(lock);
    acquire(s[i]);
}
void put_forks(int i){
    acquire(lock);
    state[i] = DENKEN;
    check_forks(LEFT(i));
    check_forks(RIGHT(i));
    release(lock);
}
void check_forks(int i){
    if(state[i]==HUNGRY && state[LEFT(i)]!=EATING && state[RIGHT(i)]!=EATING){
        state[i]=EATING
        release(s[i]);
    }
}
```

↪ Mehr Livelock da wird einer übrig

⟶ STILL STARVATION possible

# BS - KVB

→ Strategien im Umgang mit Deadlocks:

1) Ignorieren des Problems
2) Vermeidung durch konzeptionelle Maßnahmen die eine der Bedingungen für Deadlock nie eintreten lassen.
3) Erkennung und Behebung
4) Dynamische Vermeidung von Deadlocks durch vorsichtige Belegungsstrategien.

2) → Vermeidung von Deadlocks:
Schließe eine der Bedingungen aus!

↳ 1. Wechselseitiger Ausschluss: → Kann nicht umgangen werden
↳ Alternative: Nur ein Prozess darf Ressource tatsächlich anfordern.

↳ 2. Besitzen & Warten: → Prävention: Prozess fordert zu Beginn alle benötigten Ressourcen auf einmal an, nur wenn er alle Ressourcen bekommt kann er starten, sonst retry
↳ (-) Nicht alle Ressourcen bekannt & (-) Nicht effizient

↳ 3. Kein Ressourcenentzug: → Wird ein Prozess Anforderung verweigert muss er alle Ressourcen freigeben!
↳ (-) gelöschte Arbeit geht verloren or Status-Sicherung

↳ 4. zyklische Wartebedingung:
↳ Definiere lineare Ordnung auf Ressourcen
→ Ressourcen können nur in aufsteigender Reihenfolge angefordert werden. → zyklusfrei

# BS - HVB

3) → Deadlock Erkennung: Einzelne Ressource

- Jede Ressource existiert nur einmal
- Verwalte den Betriebsmittelbelegungsgraph zur Laufzeit
- Zeitpunkt der Prüfung des Graphen
  - → Bei jeder Anforderung or → Nach n Zeiteinheiten
  - or
  - → Wenn Prozessorauslastung unter einen Schwellwert sinkt.
- Enthält Graph einen Zyklus, so liegt ein Deadlock vor!

3) → DoC: Multiple Ressourcen

- Ressourcenart $r_1, \ldots, r_n$
- Ressourcenvektor $R = (e_1, \ldots, e_n)$ wobei $R_i$: Gesamtzahl der Ressourcen von Typ $r_i$
- Verfügbarkeitsvektor $(V_1, V_2, \ldots, V_n)$
  $V_i$: Noch verfügbare Anzahl Ressourcen von Typ $r_i$
- Anforderungsmatrix
  $(A_{ij})$ Prozess $i$ benötigt insgesamt $A_{ij}$ Ressourcen von Typ $r_j$
- Zuteilungsmatrix
  $(Z_{ij})$ Prozess $i$ wurde bereits $Z_{ij}$ Ressourcen von Typ $r_j$ zugeordnet
- Zusammenhänge
  - → $R_i = V_j + \sum_i Z_{ij}$
  - → $A_{ij} \leq R_j$ Prozess kann nicht mehr anfordern als vorhanden
  - → $Z_{ij} \leq A_{ij}$ Prozess bekommt nicht mehr zugeteilt als vorhanden
- Deadlock Erkennungsalgorithmus:
  - → Prüfe ob es Reihenfolge der Prozessabarbeitung gibt, sodass Prozesse in dieser Reihenfolge ohne Deadlock ablaufen können

# BS-ÜB

## Mahuny's algorithmus:

1) Initial alle Prozesse unmarkiert.
2) Wähle unmarkierten Prozess $P_i$, der zu Ende laufen kann;
   ($A_{ij} - Z_{ij} \leq V_j$ für alle $j$)
3) Falls $P_i$ existiert, so kann $P_i$ bis zum Ende laufen.
   Setze $V_j = V_j + Z_{ij}$ für alle $j$, markiere $P_i$ gelöscht
4) Sind alle Prozesse markiert, dann kein Deadlock, sonst
   liegen alle unmarkierten Prozesse im Deadlock.

**Bsp:** $R = (4\ 2\ 3\ 1)$ $\qquad V = (2\ 1\ 0\ 0)$

$$A = \begin{pmatrix} 2 & 0 & 1 & 1 \\ 3 & 0 & 1 & 1 \\ 2 & 2 & 2 & 0 \end{pmatrix} \qquad Z = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{pmatrix}$$

$$A - Z = \begin{pmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{pmatrix}$$

$$\begin{array}{rll} 2\ 1\ 0\ 0 & & \\ \hline & & P3 \\ + \ 0\ 1\ 2\ 0 & & \\ \hline 2\ 2\ 2\ 0 & & P2 \\ 4\ 2\ 2\ 1 & & P1 \\ 4\ 2\ 3\ 1 & & \\ \hline \end{array}$$

Kein Deadlock

# BS – Ü VB

## ⇝ Deadlock Behebung:

1) Durch Prozessabbruch: Alle in den Deadlock verstrickten Prozesse werden terminiert

2) Durch sukzessiven Prozessabbruch:
Am Deadlock beteiligte Prozesse werden nacheinander vom BS beendet und dann Ressourcen freigegeben, bis Anforderung der anderen Prozesse erfüllt werden können.

3) Durch teilweise Wiederholung (Checkpointing):
- Regelmäßiges Erzeugen von Checkpoints
- Checkpoint enthält Prozesszustand & Belegung der Betriebsmittel
- Liegt Deadlock vor, wähle Prozess, dessen belegte Ressource benötigt wird, und setze ihn auf Checkpoint zurück, an dem er die benötigte Ressource noch nicht inne hatte
- Die Arbeit seit letztem Checkpoint ist verloren
- Falls der erneut gestartete Prozess die Ressource wieder anfordert, muss er blockieren, bis Ressource verfügbar ist.

# BS – 4 VB

4) → Verhinderung von Deadlocks

- Ausschließe Zuteilung: & dynamische Entscheidung über Zuteilung
- ZUKUNFT!

1) Verweigerung der Prozessinitiierung.

→ Prozesse werden der Reihe nach gestartet

→ Ein neuer Prozess $P_r$ darf nur gestartet werden, wenn seine Anforderungen vollständig erfüllt werden können.

$A_{ri} - Z_{ri} \leq V_i \quad \forall r_i$

→ Alle Anforderungen werden zu Beginn gemeinsam gestellt

→ Großer Nachteil: Geringe Ueberlauffähigkeit.

2) Bauer's Algorithmus (Verweigerung der Ressourcenzuteilung)

→ Ressourcenanforderung wird nur gewährt, wenn sie garantiert nicht zu einem Deadlock führen kann

→ E in Zustand heißt sicher, falls es eine Reihenfolge der Prozesse gibt, in der vorliegenden Anforderung erfüllen werden können.

↳ sonst unsicher ↳ verwende D. E. – Alg.

→ überprüfe bei Anforderung nach Betriebsmittel, ob Zuteilung zu sicherem Zustand führt.

→ Falls ja: Teile zu

→ Falls nicht sicher: Blockiere anfordernden Prozess

→ Endet ein Prozess so, werden alle blockierte Prozesse geweckt und ihre Anforderungen gemäß 1. neu bearbeitet

# BS - KVB - Ex 3

(3-1)

a) Wann stehen zwei nebenläufige Prozesse/Threads in Konflikt?

Zwei nebenläufige Prozesse/Threads stehen im Konflikt zueinander, wenn es ein Betriebsmittel gibt, das die beiden gemeinsam benutzen.

b) Was ist ein kritischer Abschnitt (race condition)?

Ein zeitkritischer Abschnitt (race condition) ist eine Folge von Lese/Schreib-Operationen von in Konflikt stehenden Prozessen, bei denen die Korrektheit der Endzustände der Variablen abhängig von der zeitlichen Reihenfolge der Lese/Schreib-Operationen ist.

c) Wie helfen die Verfahren solche Konflikte zu vermeiden, und was sind deren Anforderungen?

Ein Verfahren zum wechselseitigen Ausschluss verhindert, dass zu einem Zeitpunkt mehr als ein Prozess lesend oder schreibend auf ein gemeinsam benutztes Betriebsmittel/Ressource kann.

⇒ Korrektheit

⇒ Fortschrittsbedingung

⇒ Fairness

⇒ Starvation & Verhungern

3-1

d) Was sind die Vor- & Nachteile von Spinlocks?

Ein Spin Lock ist eine auf Test-and-Set basierende Sperrvariable mit aktivem Warten

→ Nachteil: Aktives Warten, Starvation possible
→ Vorteil: Geeignet, wenn Warten auf Freigabe der Sperre kürzer als Kontextwechsel.

e) Was ist ein Semaphore?
Semaphor als Datenstruktur?

Ein Semaphore ist eine Datenstruktur zur Verwaltung beschränkt verfügbarer Ressourcen, auf die mehrere Prozesse oder Threads unter Vermeidung von aktivem Warten nicht zugreifen sollen.

f) Mutex?

Ein Mutex ist ein Mechanismus (Synchronisationsobjekt) zur Gewährleistung von wechselseitigem Ausschluss.

g) Deadlock?

Eine Menge M von Prozessen befindet sich in einem Deadlock, falls
→ jeder Prozess aus M auf die Freigabe von Betriebsmitteln wartet, die ein anderer Prozess aus M belegt
→ jeder Prozess aus M eine Ressource besitzt, auf deren Freigabe ein anderer Prozess aus M wartet

# TD5 - PCD

## (3-2) Mutual Exclusion

P(0)

②

blocked[0] = true

while true

P(1)

①

blocked[1] = true

while

while false

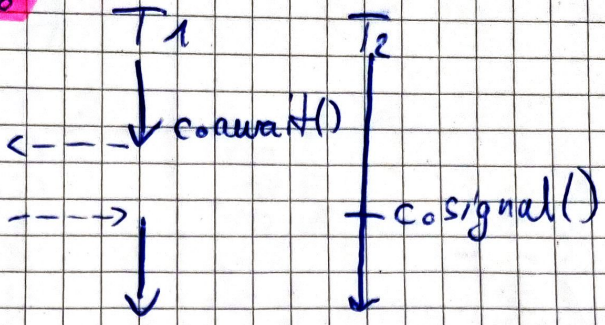
turn=0 ; blocked[2] = (false, false)

→ Seule Processus in he A.

# BS-KVS-G3

(5-3) Grayer/Vehrachter:

Condition Class:




→ When the Condition await Methods are called the lock release and before they return, the lock is reacquired and the lock hold count restored to what is was before

a) Warum werden Condition Variablen benötigt und wann sind diese mit Lock verknüpft?

Thread blockiert im kritischen Abschnitt. Damit die Wahlbedingung erfüllt werden kann, muss ein anderer Thread in den kritischen Abschnitt. Daher muss für die Zeit des Blockierens der h.A. wieder freigegeben werden. Nach dem Blockieren muss der h.A. für den Thread wieder reserviert werden.

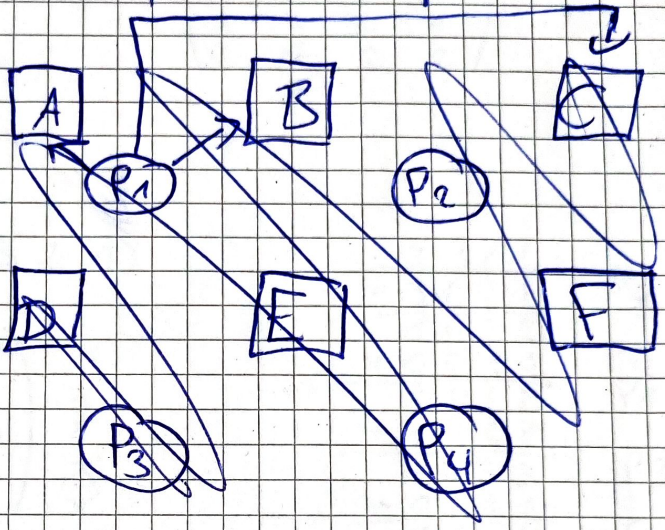
b) Was while(!==0) → if(!==0)?

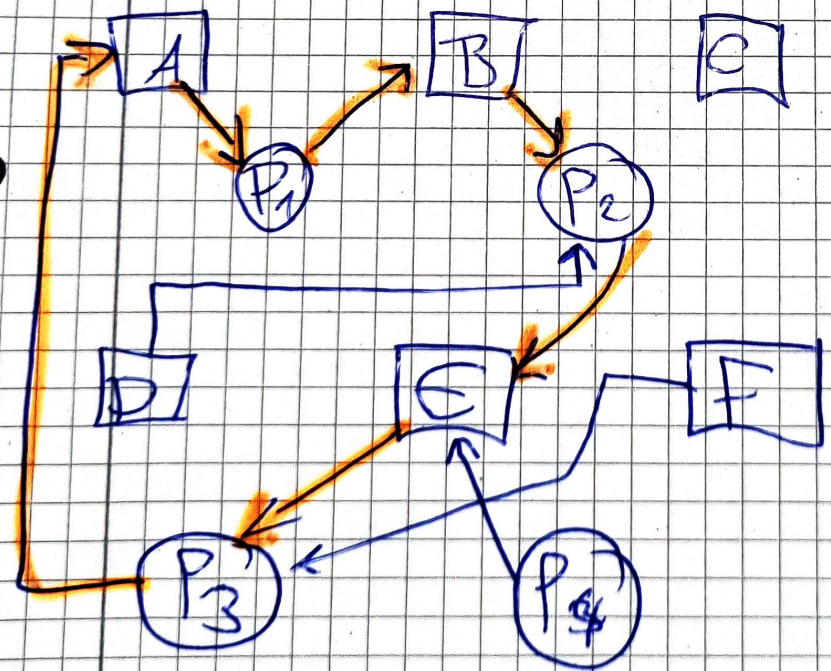
Nicht korrekt. Wenn Thread aufgeweckt wird, hat es nicht sofort Rechenzeit und ruft auch noch nicht den Mutex. Da es den Mutex erhält, kann ein anderer Thread in den h.A. eintreten und ggf. ein Element hinzufügen oder entfernen.

BS-KÜB – 6x3

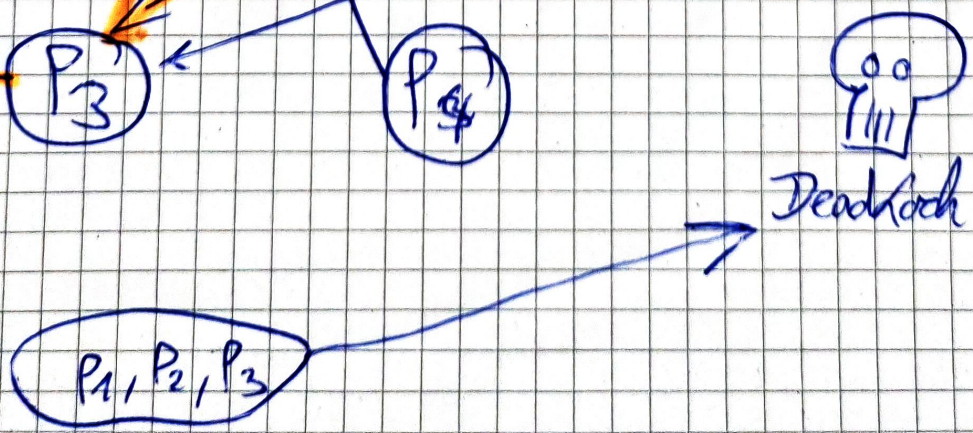
(3-4) Deadlock-Übung

| P1 | P2 | P3 | P4 |
|---|---|---|---|
| → A | → B | → E | → E |
| → D | → D | → F | → A |
| → C | → E | → A | ← A |
| ← A | ← B | ← F | → C |
| ← C | ← D | ← A | → E |
| ← B | ← E | ← E | ← C |




P1 → A
P2 → B
P1 → B
P3 → E
P2 → D
P2 → E
P3 → F
P3 → A



# BS – KUB – 6x3

## (3-9) Deadlock-Erkennung

**Zustand 1°:**

$$R = (10 \quad 9 \quad 8 \quad 7) \qquad \begin{matrix} A & B & C & D \end{matrix}$$

$$Z = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 3 & 3 & 3 & 3 \\ 0 & 2 & 2 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \qquad A = \begin{pmatrix} 3 & 4 & 3 & 2 \\ 8 & 3 & 7 & 7 \\ 5 & 5 & 5 & 4 \\ 3 & 3 & 2 & 1 \\ 4 & 1 & 2 & 2 \end{pmatrix}$$

$$\overline{Z} = (7 \quad 6 \quad 6 \quad 6)$$

$$V = R - \overline{Z} = (3 \quad 3 \quad 2 \quad 1)$$

$$A - Z = \begin{pmatrix} 2 & 4 & 3 & 1 \\ 5 & 0 & 4 & 4 \\ 5 & 3 & 3 & 3 \\ 3 & 3 & 1 & 1 \\ 1 & 0 & 2 & 1 \end{pmatrix} \begin{matrix} x \\ x \\ x \\ x \\ x \end{matrix}$$

| | | |
|---|---|---|
| 3321 | | |
| +3101 | $P_5$ | x |
| 6422 | | |
| +0010 | $P_4$ | |
| 6432 | $P_1$ | |
| +1001 | | |
| 7433 | $P_3$ | |
| +0221 | | |
| 7654 | $P_2$ | |
| 1098 | | |

↳ kein Deadlock

↳ stabil

# BS - ÜB - 6x3

## (3-8) Deadlock - Erkennung

**Zustand 2:**

$$R = (7 \quad 5 \quad 9 \quad 6)$$

$$Z = \begin{pmatrix} 2 & 0 & 3 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 5 & 1 \\ 1 & 2 & 0 & 0 \end{pmatrix} \qquad A = \begin{pmatrix} 6 & 3 & 4 & 1 \\ 2 & 2 & 1 & 3 \\ 7 & 3 & 6 & 6 \\ 7 & 3 & 1 & 6 \end{pmatrix}$$

$$\bar{Z} = (4 \quad 3 \quad 9 \quad 3)$$

$$V = R - \bar{Z} = (3 \quad 2 \quad 0 \quad 3)$$

$$A - Z = \begin{pmatrix} 4 & 3 & 1 & 0 \\ 1 & 1 & 0 & 2 \\ 7 & 3 & 1 & 5 \\ 6 & 1 & 1 & 6 \end{pmatrix} \begin{matrix} \times \\ \times \\ \\ \end{matrix}$$

| | | | | |
|---|---|---|---|---|
| | 3 | 2 | 0 | 3 |
| + | 1 | 1 | 1 | 1 | P2 |
| | 4 | 3 | 1 | 4 | |

| | | | | | |
|---|---|---|---|---|---|
| + | 2 | 0 | 3 | 1 | P1 |
| | 6 | 3 | 4 | 5 | |

↳ Deadlock auflösbar